

# Senior .NET Bug Fixing Assessment

## C# Console App .NET 8 — Factory Design Pattern

### Target Candidate

Minimum experience: 5+ years

Technology: C# 12 / .NET 8 Console Application

Design Pattern: Factory Design Pattern

Assessment Type: Bug fixing / debugging / code analysis

---

### Candidate Instructions

You are given a set of real-world style C# console application problems.

Each problem contains:

- Business scenario
- Input data
- Expected behavior
- Existing code
- Your task

The code is intentionally written like an existing production codebase.

Do not rewrite the whole application unless required. Analyze the code carefully, find the issue, and fix it with clean C# code.

Important rules:

1. Do not remove the Factory Design Pattern.
  2. Do not hardcode the final output.
  3. Do not change the business requirement.
  4. Keep the application as a .NET 8 Console App.
  5. Your fix should be maintainable and suitable for production code.
  6. Handle invalid inputs gracefully wherever needed.
-

# Question 1: Payment Gateway Factory

## Business Scenario

An online gold purchase system supports multiple payment modes.

Currently supported payment modes are:

- UPI
- Card
- NetBanking

The application receives the payment mode from the transaction record and uses a factory to create the correct payment processor.

The application works for some records but fails for certain valid payment modes received from external systems.

## Input

```
TXN001,UPI,1500
TXN002,Card,2500
TXN003,netbanking,5000
TXN004,upi,750
```

## Expected Output

```
TXN001 - UPI payment processed for 1500
TXN002 - Card payment processed for 2500
TXN003 - NetBanking payment processed for 5000
TXN004 - UPI payment processed for 750
```

## Given Code

```
using System;
using System.Collections.Generic;

namespace PaymentGatewayFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
```

```

    {
        var transactions = new List<string>
        {
            "TXN001,UPI,1500",
            "TXN002,Card,2500",
            "TXN003,netbanking,5000",
            "TXN004,upi,750"
        };

        foreach (var transaction in transactions)
        {
            var parts = transaction.Split(',');

            string transactionId = parts[0];
            string paymentMode = parts[1];
            decimal amount = decimal.Parse(parts[2]);

            IPaymentProcessor processor =
PaymentProcessorFactory.Create(paymentMode);

            string result = processor.Process(transactionId, amount);
            Console.WriteLine(result);
        }
    }

    public interface IPaymentProcessor
    {
        string Process(string transactionId, decimal amount);
    }

    public class UpiPaymentProcessor : IPaymentProcessor
    {
        public string Process(string transactionId, decimal amount)
        {
            return $"{transactionId} - UPI payment processed for {amount}";
        }
    }

    public class CardPaymentProcessor : IPaymentProcessor
    {
        public string Process(string transactionId, decimal amount)
        {
            return $"{transactionId} - Card payment processed for {amount}";
        }
    }

    public class NetBankingPaymentProcessor : IPaymentProcessor

```

```

    {
        public string Process(string transactionId, decimal amount)
        {
            return $"{transactionId} - NetBanking payment processed for
{amount}";
        }
    }

    public static class PaymentProcessorFactory
    {
        public static IPaymentProcessor Create(string paymentMode)
        {
            return paymentMode switch
            {
                "UPI" => new UpiPaymentProcessor(),
                "Card" => new CardPaymentProcessor(),
                "NetBanking" => new NetBankingPaymentProcessor(),
                _ => throw new NotSupportedException($"Payment mode
{paymentMode} is not supported")
            };
        }
    }
}

```

## Candidate Task

Fix the application so that valid payment modes are processed correctly even when data comes from external systems with different casing.

## Question 2: Report Export Factory

### Business Scenario

A back-office reporting system exports reports in different formats.

Supported report formats:

- PDF
- Excel
- CSV

The factory creates the correct exporter based on the requested format.

A new requirement says that CSV reports must include comma-separated values. However, the current output is not matching the expected report format.

## Input

```
Report Name: MonthlySales
Format: CSV
Data: Gold,Silver,Platinum
```

## Expected Output

```
MonthlySales exported as CSV: Gold,Silver,Platinum
```

## Given Code

```
using System;

namespace ReportExportFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string reportName = "MonthlySales";
            string format = "CSV";
            string[] data = { "Gold", "Silver", "Platinum" };

            IReportExporter exporter = ReportExporterFactory.Create(format);

            string output = exporter.Export(reportName, data);

            Console.WriteLine(output);
        }
    }

    public interface IReportExporter
    {
        string Export(string reportName, string[] data);
    }

    public class PdfReportExporter : IReportExporter
    {
        public string Export(string reportName, string[] data)
```

```

        {
            return $"{reportName} exported as PDF";
        }
    }

    public class ExcelReportExporter : IReportExporter
    {
        public string Export(string reportName, string[] data)
        {
            return $"{reportName} exported as Excel";
        }
    }

    public class CsvReportExporter : IReportExporter
    {
        public string Export(string reportName, string[] data)
        {
            return $"{reportName} exported as CSV: {string.Join("|", data)}";
        }
    }

    public static class ReportExporterFactory
    {
        public static IReportExporter Create(string format)
        {
            if (format == "PDF")
            {
                return new PdfReportExporter();
            }

            if (format == "Excel")
            {
                return new ExcelReportExporter();
            }

            if (format == "CSV")
            {
                return new CsvReportExporter();
            }

            throw new ArgumentException("Invalid report format");
        }
    }
}

```

## Candidate Task

Fix the implementation so that the report output matches the required business format.

---

## Question 3: Discount Calculator Factory

### Business Scenario

A billing engine calculates discounts based on customer type.

Customer types:

- Regular
- Premium
- Corporate

Discount rules:

| Customer Type | Discount |
|---------------|----------|
| Regular       | 5%       |
| Premium       | 10%      |
| Corporate     | 15%      |

The current program calculates the discount but produces an incorrect final payable amount for some customer types.

### Input

```
Customer: Premium  
Bill Amount: 10000
```

### Expected Output

```
Customer Type: Premium  
Bill Amount: 10000  
Discount Amount: 1000  
Final Amount: 9000
```

## Given Code

```
using System;

namespace DiscountFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string customerType = "Premium";
            decimal billAmount = 10000;

            IDiscountCalculator calculator =
DiscountCalculatorFactory.Create(customerType);

            decimal discountAmount = calculator.CalculateDiscount(billAmount);
            decimal finalAmount = billAmount + discountAmount;

            Console.WriteLine($"Customer Type: {customerType}");
            Console.WriteLine($"Bill Amount: {billAmount}");
            Console.WriteLine($"Discount Amount: {discountAmount}");
            Console.WriteLine($"Final Amount: {finalAmount}");
        }
    }

    public interface IDiscountCalculator
    {
        decimal CalculateDiscount(decimal billAmount);
    }

    public class RegularDiscountCalculator : IDiscountCalculator
    {
        public decimal CalculateDiscount(decimal billAmount)
        {
            return billAmount * 0.05m;
        }
    }

    public class PremiumDiscountCalculator : IDiscountCalculator
    {
        public decimal CalculateDiscount(decimal billAmount)
        {
            return billAmount * 0.10m;
        }
    }
}
```



```

public class CorporateDiscountCalculator : IDiscountCalculator
{
    public decimal CalculateDiscount(decimal billAmount)
    {
        return billAmount * 0.15m;
    }
}

public static class DiscountCalculatorFactory
{
    public static IDiscountCalculator Create(string customerType)
    {
        return customerType switch
        {
            "Regular" => new RegularDiscountCalculator(),
            "Premium" => new PremiumDiscountCalculator(),
            "Corporate" => new CorporateDiscountCalculator(),
            _ => throw new ArgumentException("Invalid customer type")
        };
    }
}

```

## Candidate Task

Fix the program so that the payable amount is calculated correctly.

## Question 4: Notification Factory with SMS and Email

### Business Scenario

A customer communication system sends notifications using different channels.

Supported channels:

- Email
- SMS

The factory is responsible for creating the correct notification sender.

The current application runs without compilation error, but one notification channel is sending content in the wrong format.

## Input

Channel: SMS  
Recipient: 9876543210  
Message: Your OTP is 456789

## Expected Output

SMS sent to 9876543210: Your OTP is 456789

## Given Code

```
using System;

namespace NotificationFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string channel = "SMS";
            string recipient = "9876543210";
            string message = "Your OTP is 456789";

            INotificationSender sender =
NotificationSenderFactory.Create(channel);

            string result = sender.Send(recipient, message);

            Console.WriteLine(result);
        }
    }

    public interface INotificationSender
    {
        string Send(string recipient, string message);
    }

    public class EmailNotificationSender : INotificationSender
    {
```

```

        public string Send(string recipient, string message)
        {
            return $"Email sent to {recipient}: {message}";
        }
    }

    public class SmsNotificationSender : INotificationSender
    {
        public string Send(string recipient, string message)
        {
            return $"SMS sent to {recipient}: {message}";
        }
    }

    public static class NotificationSenderFactory
    {
        public static INotificationSender Create(string channel)
        {
            return channel switch
            {
                "Email" => new EmailNotificationSender(),
                "SMS" => new EmailNotificationSender(),
                _ => throw new ArgumentException("Invalid notification channel")
            };
        }
    }
}

```

## Candidate Task

Analyze the factory logic and fix the application so that each channel uses the correct sender.

---

## Question 5: Shipping Charge Factory

### Business Scenario

A logistics system calculates shipping charges based on delivery type.

Delivery types:

- Standard
- Express
- SameDay

Rules:

| Delivery Type | Charge Logic    |
|---------------|-----------------|
| Standard      | Base charge 50  |
| Express       | Base charge 100 |
| SameDay       | Base charge 200 |

If the order value is above 5000, the customer should receive free Standard delivery only.

Express and SameDay should always be charged.

The current code gives free delivery for more delivery types than expected.

## Input 1

Delivery Type: Standard  
Order Value: 6000

## Expected Output 1

Shipping Charge: 0

## Input 2

Delivery Type: Express  
Order Value: 6000

## Expected Output 2

Shipping Charge: 100

## Given Code

```
using System;  
  
namespace ShippingFactoryApp  
{
```

```

internal class Program
{
    static void Main(string[] args)
    {
        string deliveryType = "Express";
        decimal orderValue = 6000;

        IShippingChargeCalculator calculator =
ShippingChargeFactory.Create(deliveryType);

        decimal shippingCharge = calculator.Calculate(orderValue);

        Console.WriteLine($"Shipping Charge: {shippingCharge}");
    }
}

public interface IShippingChargeCalculator
{
    decimal Calculate(decimal orderValue);
}

public abstract class ShippingChargeCalculatorBase :
IShippingChargeCalculator
{
    protected decimal BaseCharge { get; init; }

    public virtual decimal Calculate(decimal orderValue)
    {
        if (orderValue > 5000)
        {
            return 0;
        }

        return BaseCharge;
    }
}

public class StandardShippingCalculator : ShippingChargeCalculatorBase
{
    public StandardShippingCalculator()
    {
        BaseCharge = 50;
    }
}

public class ExpressShippingCalculator : ShippingChargeCalculatorBase
{
    public ExpressShippingCalculator()

```

```

        {
            BaseCharge = 100;
        }
    }

    public class SameDayShippingCalculator : ShippingChargeCalculatorBase
    {
        public SameDayShippingCalculator()
        {
            BaseCharge = 200;
        }
    }

    public static class ShippingChargeFactory
    {
        public static IShippingChargeCalculator Create(string deliveryType)
        {
            return deliveryType switch
            {
                "Standard" => new StandardShippingCalculator(),
                "Express" => new ExpressShippingCalculator(),
                "SameDay" => new SameDayShippingCalculator(),
                _ => throw new ArgumentException("Invalid delivery type")
            };
        }
    }
}

```

## Candidate Task

Fix the design so that free shipping is applied only for Standard delivery and not for Express or SameDay.

## Question 6: Document Verification Factory

### Business Scenario

A KYC system verifies different document types.

Supported document types:

- Aadhaar
- PAN
- Passport

Validation rules:

| Document Type | Rule                          |
|---------------|-------------------------------|
| Aadhaar       | Must be exactly 12 digits     |
| PAN           | Must be exactly 10 characters |
| Passport      | Must be at least 8 characters |

The application should validate the document based on its type.

Currently, one document type is validated incorrectly.

## Input

Document Type: PAN  
Document Number: ABCDE1234F

## Expected Output

PAN is valid

## Given Code

```
using System;

namespace DocumentVerificationFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string documentType = "PAN";
            string documentNumber = "ABCDE1234F";

            IDocumentVerifier verifier =
                DocumentVerifierFactory.Create(documentType);

            bool isValid = verifier.Verify(documentNumber);

            Console.WriteLine(isValid
                ? $"{documentType} is valid"
                : $"{documentType} is invalid");
        }
    }
}
```

```

    }
}

public interface IDocumentVerifier
{
    bool Verify(string documentNumber);
}

public class AadhaarVerifier : IDocumentVerifier
{
    public bool Verify(string documentNumber)
    {
        return documentNumber.Length == 12 && long.TryParse(documentNumber,
out _);
    }
}

public class PanVerifier : IDocumentVerifier
{
    public bool Verify(string documentNumber)
    {
        return documentNumber.Length == 12;
    }
}

public class PassportVerifier : IDocumentVerifier
{
    public bool Verify(string documentNumber)
    {
        return documentNumber.Length >= 8;
    }
}

public static class DocumentVerifierFactory
{
    public static IDocumentVerifier Create(string documentType)
    {
        return documentType switch
        {
            "Aadhaar" => new AadhaarVerifier(),
            "PAN" => new PanVerifier(),
            "Passport" => new PassportVerifier(),
            _ => throw new ArgumentException("Unsupported document type")
        };
    }
}
}

```



## Candidate Task

Fix the document validation so that each document type follows the correct business rule.

---

## Question 7: Tax Calculator Factory

### Business Scenario

An invoice system calculates tax based on product category.

Categories:

- Food
- Electronics
- Luxury

Tax rules:

| Category    | Tax |
|-------------|-----|
| Food        | 5%  |
| Electronics | 18% |
| Luxury      | 28% |

The program should calculate tax for each invoice item and print the final price.

The current result is incorrect for one category.

### Input

```
Product: Smart Watch  
Category: Electronics  
Price: 10000
```

### Expected Output

```
Product: Smart Watch  
Category: Electronics  
Price: 10000
```

Tax: 1800  
Final Price: 11800

## Given Code

```
using System;

namespace TaxCalculatorFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string productName = "Smart Watch";
            string category = "Electronics";
            decimal price = 10000;

            ITaxCalculator taxCalculator =
            TaxCalculatorFactory.Create(category);

            decimal tax = taxCalculator.CalculateTax(price);
            decimal finalPrice = price + tax;

            Console.WriteLine($"Product: {productName}");
            Console.WriteLine($"Category: {category}");
            Console.WriteLine($"Price: {price}");
            Console.WriteLine($"Tax: {tax}");
            Console.WriteLine($"Final Price: {finalPrice}");
        }
    }

    public interface ITaxCalculator
    {
        decimal CalculateTax(decimal price);
    }

    public class FoodTaxCalculator : ITaxCalculator
    {
        public decimal CalculateTax(decimal price)
        {
            return price * 0.05m;
        }
    }

    public class ElectronicsTaxCalculator : ITaxCalculator
    {

```

```

        public decimal CalculateTax(decimal price)
        {
            return price * 0.08m;
        }
    }

    public class LuxuryTaxCalculator : ITaxCalculator
    {
        public decimal CalculateTax(decimal price)
        {
            return price * 0.28m;
        }
    }

    public static class TaxCalculatorFactory
    {
        public static ITaxCalculator Create(string category)
        {
            return category switch
            {
                "Food" => new FoodTaxCalculator(),
                "Electronics" => new ElectronicsTaxCalculator(),
                "Luxury" => new LuxuryTaxCalculator(),
                _ => throw new ArgumentException("Invalid category")
            };
        }
    }
}

```

## Candidate Task

Fix the tax calculation so that the result matches the expected business rule.

## Question 8: Insurance Premium Factory

### Business Scenario

An insurance system calculates premium based on policy type.

Policy types:

- Health
- Vehicle
- Travel

Rules:

| Policy Type | Premium Calculation |
|-------------|---------------------|
| Health      | Sum insured × 2%    |
| Vehicle     | Sum insured × 3%    |
| Travel      | Sum insured × 1%    |

The current application prints a premium amount, but one policy type is routed incorrectly.

## Input

Policy Type: Vehicle  
Sum Insured: 500000

## Expected Output

Policy Type: Vehicle  
Premium: 15000

## Given Code

```
using System;

namespace InsurancePremiumFactoryApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string policyType = "Vehicle";
            decimal sumInsured = 500000;

            IPremiumCalculator calculator =
            PremiumCalculatorFactory.Create(policyType);

            decimal premium = calculator.Calculate(sumInsured);

            Console.WriteLine($"Policy Type: {policyType}");
            Console.WriteLine($"Premium: {premium}");
        }
    }
}
```

```

public interface IPremiumCalculator
{
    decimal Calculate(decimal sumInsured);
}

public class HealthPremiumCalculator : IPremiumCalculator
{
    public decimal Calculate(decimal sumInsured)
    {
        return sumInsured * 0.02m;
    }
}

public class VehiclePremiumCalculator : IPremiumCalculator
{
    public decimal Calculate(decimal sumInsured)
    {
        return sumInsured * 0.03m;
    }
}

public class TravelPremiumCalculator : IPremiumCalculator
{
    public decimal Calculate(decimal sumInsured)
    {
        return sumInsured * 0.01m;
    }
}

public static class PremiumCalculatorFactory
{
    public static IPremiumCalculator Create(string policyType)
    {
        if (policyType == "Health")
        {
            return new HealthPremiumCalculator();
        }

        if (policyType == "Vehicle")
        {
            return new VehiclePremiumCalculator();
        }

        if (policyType == "Travel")
        {
            return new TravelPremiumCalculator();
        }
    }
}

```

```
        throw new ArgumentException("Invalid policy type");
    }
}
}
```

## Candidate Task

Fix the factory so that each policy type is mapped to the correct premium calculator.

## Interviewer Answer Key

### Answer Key — Question 1: Payment Gateway Factory

#### Issue

The factory compares payment mode using exact case-sensitive strings.

External systems may send:

```
netbanking
upi
```

But the factory expects:

```
NetBanking
UPI
```

#### Corrected Code

```
public static class PaymentProcessorFactory
{
    public static IPaymentProcessor Create(string paymentMode)
    {
        string normalizedPaymentMode = paymentMode.Trim().ToUpperInvariant();

        return normalizedPaymentMode switch
        {
            "UPI" => new UpiPaymentProcessor(),
        }
    }
}
```

```

        "CARD" => new CardPaymentProcessor(),
        "NETBANKING" => new NetBankingPaymentProcessor(),
        _ => throw new
NotSupportedException($"Payment mode {paymentMode} is not supported")
    };
}
}

```

## Evaluation Points

Candidate should identify:

- Case-sensitive factory matching issue
- External system data normalization
- `Trim()` and `ToUpperInvariant()` usage
- Factory should not fail for valid data with casing difference

## Answer Key — Question 2: Report Export Factory

### Issue

CSV exporter is joining values with pipe `|` instead of comma `,`.

### Corrected Code

```

public class CsvReportExporter : IReportExporter
{
    public string Export(string reportName, string[] data)
    {
        return $"{reportName} exported as CSV: {string.Join(",", data)}";
    }
}

```

## Evaluation Points

Candidate should identify:

- Factory selection is correct
- Bug is inside concrete product implementation
- CSV format should use comma separator
- Not every factory-pattern issue is inside the factory class

## Answer Key — Question 3: Discount Calculator Factory

### Issue

The discount amount is added instead of subtracted.

Incorrect code:

```
decimal finalAmount = billAmount + discountAmount;
```

### Corrected Code

```
decimal finalAmount = billAmount - discountAmount;
```

### Evaluation Points

Candidate should identify:

- Factory returns the correct calculator
- Discount calculation is correct
- Final payable calculation is wrong
- Business logic should be verified end-to-end

---

## Answer Key — Question 4: Notification Factory

### Issue

For "SMS", the factory returns EmailNotificationSender.

Incorrect code:

```
"SMS" => new EmailNotificationSender()
```

### Corrected Code

```
public static class NotificationSenderFactory
{
    public static INotificationSender Create(string channel)
    {
        return channel switch
        {
```



```

        "Email" => new EmailNotificationSender(),
        "SMS" => new SmsNotificationSender(),
        _ => throw new ArgumentException("Invalid notification channel")
    };
}
}

```

## Evaluation Points

Candidate should identify:

- Factory is routing to the wrong concrete class
- Interface implementation is correct
- Bug is caused by incorrect object creation
- This is a common production bug during copy-paste coding

## Answer Key — Question 5: Shipping Charge Factory

### Issue

Free delivery rule is implemented in the base class.

Because `ExpressShippingCalculator` and `SameDayShippingCalculator` inherit the base implementation, free shipping is applied to all delivery types.

### Better Corrected Design

```

public abstract class ShippingChargeCalculatorBase : IShippingChargeCalculator
{
    protected decimal BaseCharge { get; init; }

    public virtual decimal Calculate(decimal orderValue)
    {
        return BaseCharge;
    }
}

public class StandardShippingCalculator : ShippingChargeCalculatorBase
{
    public StandardShippingCalculator()
    {
        BaseCharge = 50;
    }
}

```

```

    public override decimal Calculate(decimal orderValue)
    {
        if (orderValue > 5000)
        {
            return 0;
        }

        return BaseCharge;
    }
}

public class ExpressShippingCalculator : ShippingChargeCalculatorBase
{
    public ExpressShippingCalculator()
    {
        BaseCharge = 100;
    }
}

public class SameDayShippingCalculator : ShippingChargeCalculatorBase
{
    public SameDayShippingCalculator()
    {
        BaseCharge = 200;
    }
}

```

## Evaluation Points

Candidate should identify:

- Incorrect use of base class shared behavior
- Free delivery is not common behavior for all shipping types
- Override should be used only where rule differs
- Inheritance can spread wrong business logic if used carelessly

---

## Answer Key — Question 6: Document Verification Factory

### Issue

PAN validation checks for length 12.

Incorrect code:

```
return documentNumber.Length == 12;
```

PAN should be exactly 10 characters.

### Corrected Code

```
public class PanVerifier : IDocumentVerifier
{
    public bool Verify(string documentNumber)
    {
        return !string.IsNullOrEmpty(documentNumber)
            && documentNumber.Length == 10;
    }
}
```

### Better Version with Basic PAN Pattern

```
using System.Text.RegularExpressions;

public class PanVerifier : IDocumentVerifier
{
    public bool Verify(string documentNumber)
    {
        if (string.IsNullOrEmpty(documentNumber))
        {
            return false;
        }

        return Regex.IsMatch(documentNumber, "^[A-Z]{5}[0-9]{4}[A-Z]{1}$");
    }
}
```

### Evaluation Points

Candidate should identify:

- Factory mapping is correct
- Concrete verifier has the wrong validation rule
- Basic null or whitespace validation should be considered
- Senior candidate may improve PAN validation using Regex

## Answer Key — Question 7: Tax Calculator Factory

### Issue

Electronics tax is implemented as 8%, but the business rule says 18%.

Incorrect code:

```
return price * 0.08m;
```

### Corrected Code

```
public class ElectronicsTaxCalculator : ITaxCalculator
{
    public decimal CalculateTax(decimal price)
    {
        return price * 0.18m;
    }
}
```

### Evaluation Points

Candidate should identify:

- Factory selection is correct
- Concrete strategy/calculator has wrong percentage
- Business rule mismatch
- Decimal should be used for money calculations

---

## Answer Key — Question 8: Insurance Premium Factory

### Issue

The factory maps "Vehicle" to TravelPremiumCalculator.

Incorrect code:

```
if (policyType == "Vehicle")
{
    return new TravelPremiumCalculator();
}
```

## Corrected Code

```
public static class PremiumCalculatorFactory
{
    public static IPremiumCalculator Create(string policyType)
    {
        if (policyType == "Health")
        {
            return new HealthPremiumCalculator();
        }

        if (policyType == "Vehicle")
        {
            return new VehiclePremiumCalculator();
        }

        if (policyType == "Travel")
        {
            return new TravelPremiumCalculator();
        }

        throw new ArgumentException("Invalid policy type");
    }
}
```

## Cleaner Version

```
public static class PremiumCalculatorFactory
{
    public static IPremiumCalculator Create(string policyType)
    {
        return policyType.Trim().ToUpperInvariant() switch
        {
            "HEALTH" => new HealthPremiumCalculator(),
            "VEHICLE" => new VehiclePremiumCalculator(),
            "TRAVEL" => new TravelPremiumCalculator(),
            _ => throw new ArgumentException("Invalid policy type")
        };
    }
}
```

## Evaluation Points

Candidate should identify:

- Wrong concrete class returned from factory
- Factory routing bug
- Case normalization can make the factory more robust
- Copy-paste bugs are common in factory implementations

## Overall Scoring Rubric

| Evaluation Area               | What to Check                                                                     |
|-------------------------------|-----------------------------------------------------------------------------------|
| Factory Pattern Understanding | Candidate knows factory creates concrete objects based on input                   |
| Debugging Skill               | Candidate can locate whether issue is in factory, concrete class, or calling code |
| Business Rule Analysis        | Candidate compares code behavior with expected output                             |
| Clean Fix                     | Candidate avoids unnecessary rewrites                                             |
| C# Quality                    | Candidate uses proper naming, decimal for money, clean switch expressions         |
| Robustness                    | Candidate handles casing, trimming, invalid inputs                                |
| Senior-Level Thinking         | Candidate suggests maintainable improvements without over-engineering             |

## Suggested Interview Follow-Up Questions

1. What problem does the Factory Design Pattern solve?
2. When should we use Factory Pattern instead of directly using `new`?
3. What is the difference between Factory Method and Simple Factory?
4. What happens if the factory grows with too many `if` or `switch` cases?
5. How can dependency injection replace or improve factory usage?
6. How do you unit test a factory class?
7. Should business rules be placed inside the factory?
8. What is the risk of copy-paste mapping inside factories?
9. How can enum-based input improve this code?
10. How can we avoid string-based factory keys?
11. How would you add a new payment mode without modifying too many files?
12. How can factory pattern violate the Open/Closed Principle if not designed properly?

---

## Senior-Level Enhancement Ideas

For stronger candidates, ask them to improve one of the problems using:

- Enum-based factory key
  - Dictionary-based factory registration
  - Dependency Injection
  - Generic factory
  - Reflection-based factory with caution
  - Unit tests using xUnit or NUnit
  - Interface segregation
  - Open/Closed Principle
- 

## Sample Improved Factory Using Dictionary Registration

```
public static class PaymentProcessorFactory
{
    private static readonly Dictionary<string, Func<IPaymentProcessor>>
Processors =
        new(StringComparer.OrdinalIgnoreCase)
        {
            ["UPI"] = () => new UpiPaymentProcessor(),
            ["Card"] = () => new CardPaymentProcessor(),
            ["NetBanking"] = () => new NetBankingPaymentProcessor()
        };

    public static IPaymentProcessor Create(string paymentMode)
    {
        if (string.IsNullOrEmpty(paymentMode))
        {
            throw new ArgumentException("Payment mode is required");
        }

        if (Processors.TryGetValue(paymentMode.Trim(), out var
processorFactory))
        {
            return processorFactory();
        }

        throw new NotSupportedException($"Payment mode {paymentMode} is not
```

```
supported");  
    }  
}
```

## Why This Version Is Better

- Avoids long switch blocks
- Uses case-insensitive matching
- Easier to add new processors
- Cleaner and more maintainable
- Better for real-world production code

---

## Final Trainer Note

These questions are intentionally designed so that the bug is not always inside the factory class.

In real projects, Factory Pattern issues may appear in:

- Factory selection logic
- Wrong concrete class mapping
- Concrete implementation behavior
- Caller-side usage
- Shared base class logic
- Input normalization
- Business rule mismatch

A strong 5+ years candidate should not assume that the factory itself is always wrong. They should trace the flow from input to factory creation, concrete implementation, and final output.